

TECHNICAL WALKTHROUGH

How Minting & Burning Actually Works

A step-by-step explanation of the on-ramp / off-ramp token flow, as currently built

Solana SPL tokens

Mint on fiat-in

Burn on fiat-out

Company: NivixPe Private Limited

Scope: Current (legacy) bridge-service implementation

Date: August 2026

FOR INTERNAL / TECHNICAL DISCUSSION

Contents

1. The Core Idea, in One Sentence

2. Who's Involved

3. Mint Side — Fiat In → Token Created

4. Burn Side — Token Destroyed → Fiat Out

5. The Full Flow, Start to Finish

6. What the "USD Token" Actually Means Today

7. Current Gaps — What's Not Really Wired Up

8. Where This Is Headed

A. Appendix — Glossary

1. The Core Idea, in One Sentence

The token is not currency. It's a temporary receipt that exists only for the few seconds between "fiat went in" and "fiat came out." Nobody is meant to hold it, save it, or spend it like money — it's just the on-chain hop in the middle of a transaction that starts and ends in real bank money.

Two operations make up the whole lifecycle of that receipt:

Operation	What it means	Who triggers it
Mint	Create brand-new tokens out of nothing and place them in a wallet	Nivix's backend (it holds the "mint authority" key)
Burn	Permanently destroy tokens, removing them from supply forever	The wallet owner (only they can authorize destroying their own balance)

Every transaction does exactly one mint and one burn, in sequence, for the same amount. Supply goes up when fiat comes in; supply comes back down when fiat goes out. If the system is working correctly, total tokens in existence at any moment should represent money that is "in transit" — already collected from a sender, not yet paid to a recipient.

2. Who's Involved

Party	Role in this flow
Sender	Pays real fiat money (e.g. INR) into the system
Payment gateway (Razorpay / Cashfree)	Actually collects the sender's payment into Nivix's bank account; confirms via webhook. <i>Only handles INR today.</i>
Nivix backend (bridge-service)	Decides how many tokens to mint, builds and signs the mint transaction, later verifies the burn and triggers payout
Treasury wallet	A single Solana keypair that holds "mint authority" over all 7 currency-named tokens — it can create unlimited units of any of them
User's Solana wallet (Phantom, etc.)	Receives the minted tokens; later signs the burn instruction to destroy them
Solana network	Records the mint and the burn as public, verifiable transactions (~2 seconds each)
Payout provider (Cashfree Payouts)	Pays the recipient real fiat after the burn is verified. <i>Only handles INR payout today.</i>
Recipient	Receives real fiat money out of the system

3. Mint Side — Fiat In → Token Created

- 1 **Sender pays.** They enter an amount (e.g. ₹8,300) in the app and confirm payment.

SENDER · INSTANT

- 2 **Payment gateway collects it.** Razorpay/Cashfree takes the ₹8,300 into Nivix's own bank account. This step only accepts INR — hardcoded in the payment gateway integration.

RAZORPAY / CASHFREE · INSTANT

- 3 **Webhook confirms payment.** The gateway notifies Nivix's backend: "₹8,300 received for order #123."

PAYMENT GATEWAY → NIVIX BACKEND · SECONDS

- 4 **Backend calculates the token amount.** Using an exchange rate (e.g. ₹83 = 1 unit), it decides how many tokens to create — say, 100.

NIVIX BACKEND · INSTANT

- 5 **Backend mints the tokens.** It builds a Solana transaction containing a `MintTo` instruction, creates the user's token account if needed, and signs it with the treasury's private key — the only key allowed to create new units of this token.

NIVIX BACKEND + TREASURY KEY · ~2 SECONDS

- 6 **Transaction confirms.** The user's wallet now shows 100 tokens that did not exist a moment ago. Total supply of that token has permanently increased by 100.

SOLANA NETWORK · ~2 SECONDS

Worth being direct about: "minting" here means creating value from nothing, authorized by a single private key. Nothing on-chain proves that real fiat backing exists to match what gets minted — that trust currently depends entirely on Nivix's internal bookkeeping being correct and the treasury key never being misused or compromised.

State after step 6: ₹8,300 sits in Nivix's bank account (real money). 100 tokens sit in the user's Solana wallet (created by Nivix, backed by nothing on-chain). Nothing has reached a recipient yet.

4. Burn Side — Token Destroyed → Fiat Out

7 User initiates withdrawal. They choose a recipient and confirm the transfer. Their own wallet (Phantom) signs a Burn instruction for those 100 tokens.

USER'S OWN WALLET · INSTANT

8 Burn confirms on-chain. The 100 tokens are permanently destroyed. Total supply drops back down by exactly the amount minted in step 6.

SOLANA NETWORK · ~2 SECONDS

9 Backend verifies the burn. The frontend sends the burn's transaction signature to Nivix's backend, which checks Solana directly to confirm the burn actually happened and wasn't faked.

NIVIX BACKEND · SECONDS

10 Backend triggers the payout. Once verified, it calls the payout provider: "pay ₹8,282 (after fees) to this recipient's bank account."

NIVIX BACKEND → CASHFREE · SECONDS

11 Recipient is paid. Cashfree either wires the amount directly to a bank account, or sends a claim-link the recipient redeems to their bank account or UPI.

CASHFREE · MINUTES–HOURS

12 Confirmation shown. Both sender and recipient see a completed transaction with a receipt in the app.

NIVIX APP · INSTANT

Why the burn is signed by the user, not the backend: the SPL Token program requires the token account's owner to authorize destroying their own balance. Only the treasury can *create* tokens (mint authority); only the holder can *destroy* their own tokens. That asymmetry is a property of the SPL Token standard itself, not a Nivix design choice.

5. The Full Flow, Start to Finish



The token exists for only a few seconds, mid-transaction — a bridge between two fiat events, never something a user is meant to sit on or accumulate.

6. What the "USD Token" Actually Means Today

The system has **7 separate tokens**, one labeled per currency (USD, EUR, INR, GBP, JPY, CAD, AUD). This is the part that causes the most confusion, so it's worth being precise:

These are not real currencies on-chain. They are ordinary SPL tokens that Nivix's own code created and named after currencies. Nothing backs them. The treasury wallet is the mint authority for all seven — it can create unlimited units of any of them, of any currency label, at zero cost, with no proof of matching real-world reserves.

6.1 What was actually tested

In the transaction demonstrated so far, **both sender and recipient were in India**. The flow was:

Real ₹ IN → mint "USD"-labeled tokens → burn them → real ₹ OUT
(India) (misleadingly named) (India)

No dollars were involved anywhere. No money crossed a border. The token happened to be named "USD" because that's the default destination-currency slot the code fills in — not because any US-side rail exists.

6.2 The other 6 tokens

Token	Payment rail behind it	Status
USD	None (domestic INR-only bridge, mislabeled)	Only one exercised in testing
INR, EUR, GBP, JPY, CAD, AUD	None — no collection or payout integration exists for any of them	Unused / decorative

These six exist only as mint addresses in a configuration file and as options in a frontend dropdown. Selecting them in the UI does not fail loudly — it just has nothing real behind it.

7. Current Gaps — What's Not Really Wired Up

Gap	Detail
Payment collection is INR-only	The Razorpay integration explicitly rejects any currency other than INR
Payout is India-only	Cashfree/RazorpayX/PayU/Instamojo are all domestic Indian payout rails — none can reach a foreign bank account
No reserve proof	Nothing on-chain shows that minted tokens are backed by real fiat sitting in Nivix's account — this depends entirely on internal bookkeeping being correct
Single point of trust	One private key (the treasury keypair) has unlimited mint authority over all 7 tokens — if it's compromised, unlimited fake tokens can be created
Server-side burn path is broken	A fallback path exists for the backend to burn on a user's behalf, but it signs with the wrong key and would be rejected on-chain if actually invoked — in practice, burns only work because the user's own wallet signs client-side
Dead UI pages	Two frontend pages (PaymentApp, ComprehensiveTesting) implement a full multi-currency picker but their routes silently redirect elsewhere — unreachable in the running app

8. Where This Is Headed

The plan already locked in for the rebuild replaces all 7 self-minted tokens with **one real asset: Circle's USDC** — removing the two biggest problems at once.

	Today (7 self-minted tokens)	Target (USDC)
Who can create it	Nivix's own treasury key — unlimited, no oversight	Circle only — regulated, audited issuer
Is it backed?	No on-chain proof; trust = internal bookkeeping	Publicly attested 1:1 reserves, audited monthly
How many tokens exist	7 (6 of them entirely decorative)	1 — used for every corridor, every currency pair
What currency labels mean	Confusing — "USD"/"INR" tokens are not real dollars/rupees	No on-chain currency tokens at all — INR/USD/EUR stay off-chain, handled by licensed partners; USDC is just the settlement rail in the middle

Same shape, real asset: the 12-step flow in Section 5 doesn't change conceptually. Step 5 ("mint from nothing") becomes "release real, already-funded USDC from the treasury." Step 8 ("burn") becomes "USDC moves back to the treasury or onward to the next hop." The fiat legs at either end — collection and payout — stay exactly as they are; they just connect to one trustworthy asset instead of seven fabricated ones.

Appendix — Glossary

SPL Token

Solana's standard for creating custom tokens — anyone can create one; whoever holds the "mint authority" key can create new units of it.

Mint authority

The private key authorized to create new units of a specific token. In this system, one treasury key holds mint authority over all 7 currency-named tokens.

Mint (verb)

The act of creating new tokens and assigning them to a wallet. Increases total supply.

Burn

The act of permanently destroying tokens from a wallet. Decreases total supply. Must be authorized by the token account's owner.

Associated Token Account (ATA)

The specific on-chain account where a wallet's balance of one particular token is stored.

Treasury wallet

The Solana keypair Nivix's backend controls, used here as both the mint authority and the fee-payer for transactions.

USDC

A real, regulated stablecoin issued by Circle, redeemable 1:1 for US dollars — the asset intended to replace all 7 self-minted tokens.